

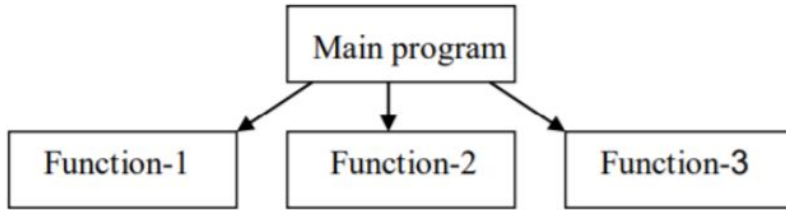
Outlines

- POP and OOP
- Concepts of OOP
- Principles/Features of OOP
- Benefits of OOP
- Applications of OOP

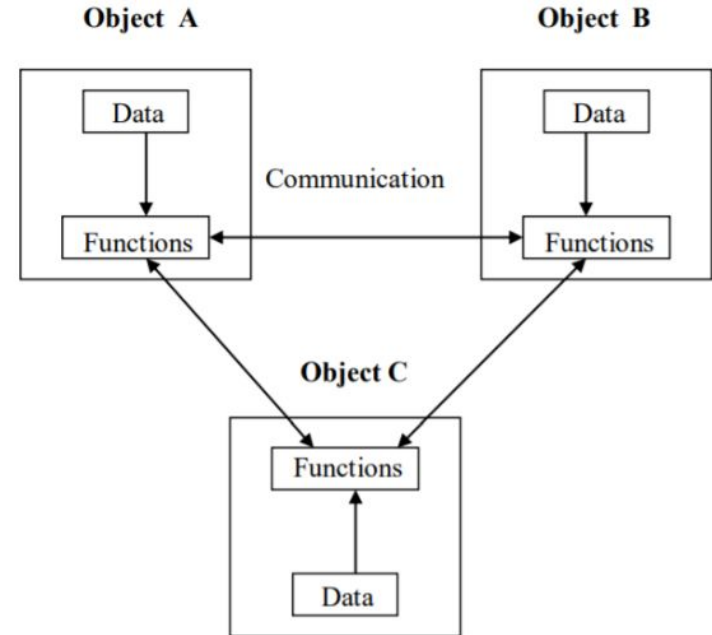
Google Classroom Code: **cbp4rds**

The high-level programming languages are broadly categorized into two categories:

Procedure oriented programming (POP) languages



Object oriented programming (OOP) language.



Procedure Oriented Programming (POP)

- In “**procedural**” programming, problems are “decomposed” into smaller units of repeatable activities called **procedures**.
- Use a **series of instructions or statements** which are **sequential** from beginning to the end.
- POP basically consists of writing a list of **instructions** for the computer to follow and organizing these instructions into groups called **functions**.
- More focused to **functions** and less to **data**
- Examples:
 - C, COBOL, Fortran, LISP, Perl

Disadvantages

Focus on processes

- Importance is given to the operation on data rather than the data.

No information hiding

- Data is exposed to the whole program.

Difficult to manage large programs

- Imagine changing the type of a global variable from int to long in a large program!!

Difficult to relate with real-world objects

- In physical world, we deal with objects such as people, cars etc. Such objects have both attributes (data) and behavior (functions).
- Neither data nor functions, by themselves, model real-world objects effectively.

Object Oriented Programming (OOP)

- Relies on the concept of **objects** and **classes**.
- The fundamental idea:
 - Combine into a single unit both data (attributes) and the functions that operate on that data (functionality/behavior).
- In this paradigm, a computer program is built with a collection of reusable components called objects.
- Related piece of information and behaviours are organized together into a template called a **class**.
- A class is thus a description of a number of **similar objects**.
- **An object is an instance of a class.**

OOP Basic Terminology

Object: Any entity that has state and behavior is known as an object. Usually a person, place or thing. For example: chair, pen, table, keyboard, bike etc.

Method: An action performed by an object

Property or attribute: Characteristics of certain object.

Class: A collection of similar objects.

For example:

In real life, a **car** is an **object**.

- The car has attributes, such as **weight and color**, and methods, such as **drive and brake**.
- Attributes and methods are basically **variables** and **functions** that belongs to the class.
- often referred to as "**class members**".

Car class

Model, Price, Color, Build
year



Objects



Model: AAA
Price: 10K
Color: Orange
Build year: 2015



Model: BBB
Price: 15K
Color: Blue
Build year: 2018



Model: CCC
Price: 45K
Color: Green
Build year: 2015

Class Name : Car

Attributes:

- Model
- Price
- Color
- Build Year

Methods:

- drive()
- brake()

Objects:

- car1
- car2
- car3

Structure in C

```
#include <stdio.h>
#include <string.h>

struct Car {
    char model[20]; // Attributes
    int price;
    char color[20];
    int build_year;
};

void drive() { // Function to simulate driving
}

int main() {
    struct Car car1, car2, car3; // Create Car objects
    strcpy(car1.model, "AAA");
    car1.price = 10;
    strcpy(car1.color, "Orange");
    car1.build_year = 2015;

    // Print attribute values
    printf("Car 1:\n");
    printf("  Model: %s\n", car1.model);
    printf("  Price: %d\n", car1.price);
    printf("  Color: %s\n", car1.color);
    printf("  Build Year: %d\n", car1.build_year);

    return 0;
}
```

Class in C++

```
#include <iostream>
using namespace std;

class Car{           // Class
public:              // Access specifier
    string model; // Attributes
    int price;
    string color;
    int build_year;

    void drive(){ // Method/Action performed by car
    }

};

int main() {
    Car car1, car2, car3; // Create objects of Car
    car1.model = "AAA";
    car1.price = 10;
    car1.color = "Orange";
    car1.build_year = 2015;

    //Print attribute values
    cout<<car1.model<<endl;
    cout<<car1.price<<endl;
    cout<<car1.color<<endl;
    cout<<car1.build_year<<endl;

    return 0;
}
```



Class →

↑
Object

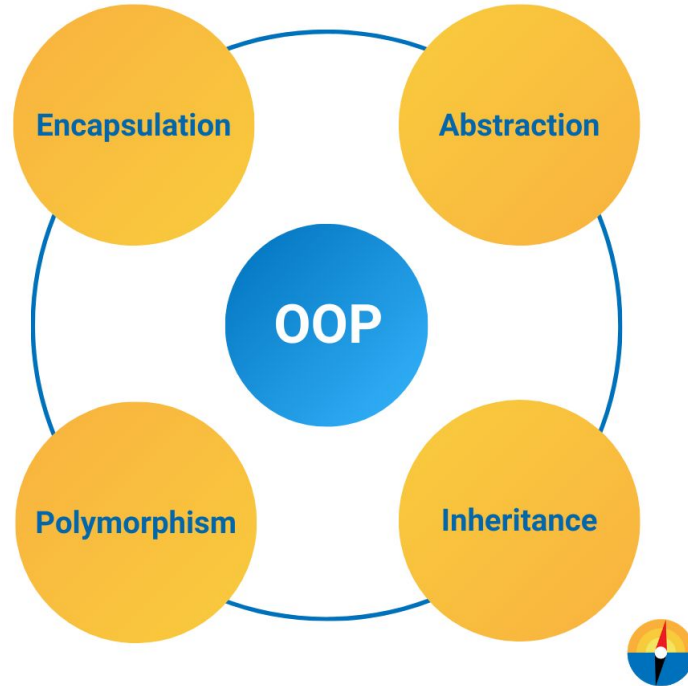
Pokemon
Name: Pikachu
Type: Electric
Health: 70
attack()
dodge()
evolve()



Attributes

Methods

Principles/Features of OOP



Encapsulation

Data encapsulation and data hiding are among the key concepts in OOP.

Data encapsulation:

- Bundling up of data with the methods that operate on that data.

Data hiding:

- Restricting direct access to the data (so that it is safe from accidental alteration).
- Only those functions which are associated with the object can access data of that object.
- However, function of an object can access the functions of other objects.
(Calling an object's function is referred to as sending a message to the object)
- An object cannot modify the data of another object directly but can do so indirectly through the functions.

Abstraction

Providing only essential information to the outside world and hiding their background details.

Its main goal is to handle complexity by hiding unnecessary details from the user.

Examples:

- While using mobile phone, we know what happens when we click on home button, back button and many more. But unknown about internal working

mechanism.

- Your program can make a call to the `sort()` function without knowing which algorithm the function actually uses to sort the given values.

Abstraction separates code into interface and implementation.

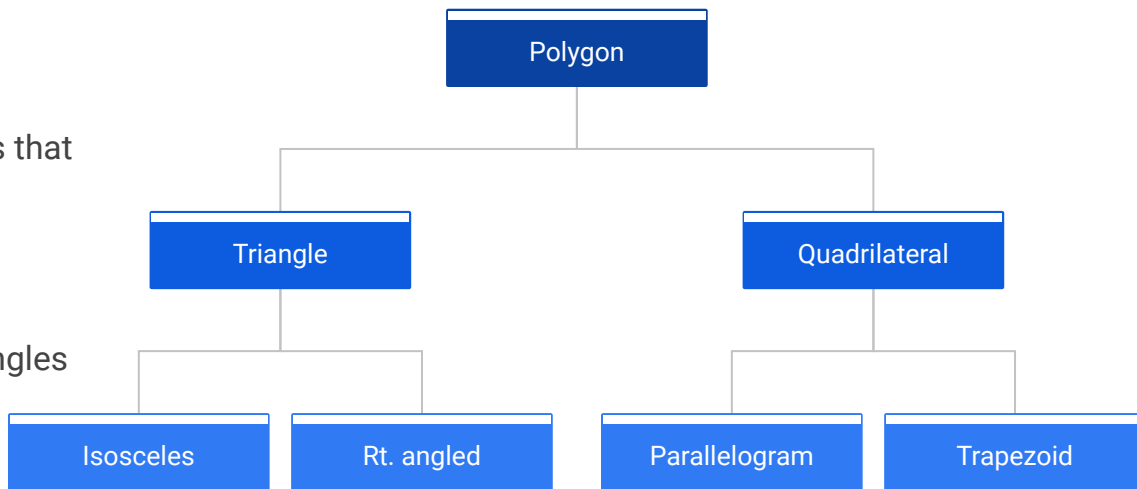
Interface should be independent of the implementation so that if the underlying implementation is changed, the interface would remain intact.

Inheritance

- Capability of a class to derive properties and characteristics from another class.
- Helps to add the additional features in an existing class without modifying it.
- Supports the concept of hierarchical classification.

- Example:

- Suppose we have a class called Polygon.
- Triangle and Quadrilateral are two classes that inherit some properties of polygon, and add some properties of their own.
- Similarly, Isosceles, and Right-angled triangles are triangles with special properties.



Polymorphism

- Poly = many
- Morph = forms
- Polymorphism: Ability to take more than one form. Processing objects differently based on their data type.
- Examples:
 - + operator will generate sum when two operands are numbers but performs string concatenation and produces third string if the operands are strings.
 - In a car with a gear transmission system, when the engine is accelerated then depending upon which gear is engaged different amount power and movement is delivered to the car.

Benefits of OOP

- Reusability

- Once a class has been written, created, and debugged, it can be distributed to other programmers for use in their own programs.

- Robustness

- Most object-oriented languages support exception and error handling

- Extensibility

- A programmer can take an existing class and, without modifying it, add additional features and capabilities to it (thanks to the inheritance).

- Easier to manage

- Each object is relatively small, self-contained, and manageable, thus reducing the complexity of the program.

Applications of OOP

- User interface design such as windows, menu.
- Real – Time systems.
- Simulation and modeling.
- Object oriented databases.
- AI and expert systems.
- Neural networks and parallel programming.
- Decision support and office automation systems.
- CIM / CAM / CAD system.



Any Queries?